

Solving the Temporal Knapsack Problem via Recursive Dantzig-Wolfe Reformulation

Alberto Caprara

University of Bologna, Italy

Fabio Furini

LAMSADE, Université Paris-Dauphine 75775 Paris, France

Enrico Malaguti

DEI, University of Bologna 40136 Bologna, Italy

Emiliano Traversi

LIPN, Université Paris 13 93430 Villetaneuse, France

Abstract

The Temporal Knapsack Problem (TKP) is a generalization of the standard Knapsack Problem where a time horizon is considered, and each item consumes the knapsack capacity during a limited time interval only. In this paper we solve the TKP using what we call a Recursive Dantzig-Wolfe Reformulation (DWR) method. The generic idea of Recursive DWR is to solve a Mixed Integer Program (MIP) by recursively applying DWR, i.e., by using DWR not only for solving the original MIP but also for recursively solving the pricing sub-problems. In a binary case (like the TKP), the Recursive DWR method can be performed in such a way that the only two components needed during the optimization are a Linear Programming solver and an algorithm for solving Knapsack Problems. The Recursive DWR allows us to solve Temporal Knapsack Problem instances through computation of strong dual bounds, which could not be obtained by exploiting the best-known previous approach based on DWR.

Keywords: Temporal Knapsack Problem ; Dantzig-Wolfe Reformulation ; Mixed Integer Programs

1 Introduction

In this paper we solve the Temporal Knapsack Problem (TKP) using what we call a Recursive Dantzig-Wolfe Reformulation (DWR) method. The TKP is a generalization of the standard Knapsack Problem where a time horizon is considered and each item consumes the knapsack capacity during a limited time interval only. The knapsack capacity cannot be exceeded at any

time instant. We compare the Recursive DWR method to the “natural” formulation of the TKP as a Mixed-Integer Program (MIP) and to the best previous approach based on DWR.

DWR of a MIP is usually tackled through *column generation*, and it requires solving a number of pricing sub-problems which may be modeled again as MIPs of smaller sizes. In a standard DWR one would tackle the sub-problems by exploiting their special structure, or as simple MIPs through a general purpose solver.

In this work we investigate instead the idea of solving the TKP by recursively applying DWR. We move from the consideration that if DWR is faster than a general purpose solver in solving some specific families of MIPs, as it is the case for the TKP (see Caprara, Furini and Malaguti [8]), then it can be recursively applied in order to achieve incremental computing time reduction. In the DWR of the TKP, the pricing subproblems are TKPs of smaller sizes (see [8]). The Recursive DWR method can be applied until the reformulation results either into single-constraint sub-problems (i.e., “simple” knapsack problems) or into smaller TKP sub-problems (“easier” to solve). Finally, note that not only for the original MIP but also for each intermediate step of the procedure we are interested in *integer* solutions, and thus the overall method needs to be integrated in a branching scheme, thus resulting in a Branch-and-Price algorithm (B&P).

The TKP is formally defined as follows. A set of n items is given, the i -th of which has size w_i , a profit p_i , and is *active* only during a time interval $[s_i, t_i)$. A subset of items has to be packed into a knapsack of capacity C such that the total profit is maximized and the knapsack capacity is not exceeded at any point in time. In order to satisfy the previous requirement, it is enough to impose that a capacity constraint is satisfied at the start time s_i of each item i . Let $S_j := \{i : s_i \leq j \text{ and } j < t_i\}$ denote the set of active items at time j , and x_i a binary variable equal to 1 if item i is selected. A binary program for the problem reads:

$$\max \left\{ \sum_{i=1}^n p_i x_i : \sum_{i \in S_j} w_i x_i \leq C, j = s_1, \dots, s_n, x_i \in \{0, 1\}, i = 1, \dots, n \right\}. \quad (1)$$

In the TKP each variable appears with the same positive coefficient in a subset of consecutive constraints only. In addition, the TKP has the property that, by selecting any subset of the constraints, one still gets a TKP. By noting that some of the capacity constraints may be dominated and hence redundant, we define by $N \subseteq \{1, \dots, n\}$ as the index subset of the non-dominated constraints (see [8] for further details).

Literature review We review papers dealing with the TKP and then we contextualize the Recursive DWR comparing it with previous related approaches.

As far as the TKP is concerned, Arkin and Silverberg [1] consider the problem of scheduling jobs with predefined starting and ending times on C identical machines, by maximizing the value of the completed jobs, where each machine can process one job at a time. This corresponds to a polynomial-time solvable TKP in which all the requests are identical. Chen, Hassin, and Tzur [7] sketch a dynamic programming approach for the TKP. In addition, they propose a

polynomial-time approximation algorithm for the special case of the problem in which the profit of an item i is proportional to $(t_i - s_i)w_i$. Calinescu et al. [6] show a randomized polynomial-time approximation algorithm for the TKP with guarantee arbitrarily close to 2. Bansal et al. show that the restricted case of TKP in which the item weights and the capacity are integers in $[0, L]$ and $L \leq 2^{\text{polylog}(n)}$, has a polynomial-time approximation scheme. The result holds for the generalization of TKP in which the available capacity is not necessarily a constant C but may vary over time. Darmann, Pferschy, and Schauer discuss some generalizations of the problem, review complexity results for special cases of TKP, and prove that the problem remains NP-hard for the case in which all the items' profits are identical. Finally, the *strong* NP-hardness of TKP was proved by Bonsma, Schulz, and Wiese [5]. The existence of a polynomial-time approximation scheme remains open. The main computational investigation on the TKP is by Caprara, Furini and Malaguti [8], where the problem is solved by DWR, and the subproblems are associated with groups of constraints. In the paper, the strength of alternative reformulations, obtained by varying the sizes of the groups, are considered.

As far as the idea of recursively applying DWR is concerned, related approaches (denoted as “nested decomposition”) have been considered for linear programs having a special structure. In a so-called multistage linear program, the constraints can be partitioned into an ordered set of stages, in such a way that each variable appears with a non-zero coefficient in the constraints of at most two consecutive stages. Hence, the constraint matrix has a staircase structure and these problems are also known as staircase linear programs. Glassey [12] and Ho and Manne [13] present nested DW decomposition schemes for multistage linear programs, where each stage operates as a master problem for the following stage, and as a subproblem for the preceding. Local and global optimality conditions and solution procedures are discussed in the papers. In a dual perspective, Wittrock [16] applies a similar scheme to the dual of a multistage linear program.

Notice instead that the constraint matrix of the TKP does not have a staircase structure as the one defined in [12, 13, 16], since potentially a variable can appear in all the constraints. In the method we propose, at each level of the recursion, we do not have a stage hierarchy in which a stage acts as a subproblem for the preceding one. Instead, all subproblems are independently solved, and the overall coherence for the value of common variables is guaranteed by a master problem (see next section for further details). Finally, we do not restrict to linear programs, but explicitly consider the integrality of the variables, and embed our decompositions in a branching scheme, thus defining a Branch-and-Price algorithm.

2 The Recursive DWR method

We review the main concepts of DWR applied to the TKP following the notation of [8]. For more details on DWR and column generation we refer the reader to Desaulniers et al. [9] and Vanderbeck and Wolsey [15].

Given a subset $Q \subseteq N$ let:

$$\mathcal{P}_Q := \text{conv} \left\{ x \in \{0, 1\}^{\bigcup_{j \in Q} S_j} : \sum_{i \in S_j} w_i x_i \leq C, j \in Q \right\}$$

denote the convex hull of the feasible 0-1 solutions to the capacity constraints in Q , $\mathcal{V}_Q$ the set of vertices of $\mathcal{P}_Q$, and y_v^Q for $v \in \mathcal{V}_Q$ the coefficients in a convex combination of the vertices in $\mathcal{V}_Q$. One may apply separately this reformulation to any collection $\mathcal{Q}$ of subsets of N . Assuming for simplicity that $\mathcal{Q}$ covers N (i.e., each $j \in N$ belongs to some $Q \in \mathcal{Q}$), this leads to the overall DWR reformulation of TKP:

$$\max \left\{ \sum_{i=1}^n p_i x_i, x_i = \sum_{v \in \mathcal{V}_Q} v_i y_v^Q, Q \in \mathcal{Q}, i \in \bigcup_{j \in Q} S_j, \sum_{v \in \mathcal{V}_Q} y_v^Q = 1, Q \in \mathcal{Q}, x_i \in \{0, 1\}, i = 1, \dots, n, y_v^Q \geq 0, Q \in \mathcal{Q}, \right.$$

Since there is an exponential number of y_v^Q variables for each $Q \in \mathcal{Q}$ and $v \in \mathcal{V}_Q$, column generation techniques are necessary to solve the linear programming relaxation of the formulation. The variables with a positive reduced profit can be generated by solving a suited TKP with constraints restricted to subset Q and defining as profits the optimal values of the associated dual variables (see [8]).

Caprara et al. [8] considered the DWR for the TKP, and investigated the relation between the strength of the bound of the corresponding linear relaxation and the size of the blocks of constraints. As one may expect, experiments showed that increasing the size of the blocks, that is, reducing the number of blocks, improves the strength of the linear relaxation bound but, at the same time, determines sub-problems which are more difficult to solve. One of the strongest linear relaxation bounds that can be obtained is the one obtained by splitting the constraints into two blocks only (having half of the constraints in the first block and half in the second). In the following, we denote this bound as the *two-block bound*.

The Recursive DWR solves the linear programming relaxation of the TKP pricing problems by recursively applying the DWR. In order to obtain integer solution at each level of the recursion, a branching scheme is enforced. Notice that the quality of the linear programming bound obtained at the root node of the branching scheme depends uniquely on the initial partition of the constraints. Since at each level of recursion the Recursive DWR method splits the TKP constraints into two or more blocks and imposes integrality on the variables, potentially the Recursive DWR method can lead to sub-problems of one constraint only, or the recursion can be stopped after a given number of levels leading to TKP subproblems of multiple constraints.

In general, there are two main design decisions to be taken in order to perform the Recursive DWR method: (i) how to decompose each level, that is, how many blocks, and which constraint goes in each block; (ii) how many levels of recursion to apply before using a MIP solver. These decisions can be represented via a rooted tree, called *Sub-problem Tree* in the following, where each node is a sub-problem associated with a subset of the constraints of its father node. Hence, the root node represents the original problem and the child nodes of the root node represent

the first level of sub-problems, their child nodes represent the second level and so on until the last level of sub-problems is represented by the *leaves*. In this paper, we consider sub-problem trees that are *binary* (at each level only two blocks are created) and *balanced* (each of the two blocks has approximatively the same size). At each node, the two sets of constraints associated with its child nodes represent a *complete partition* of the father constraint set. Since this scheme is applied at each level of the recursion, at the root node the linear programming relaxation corresponds to the two-block bound. We call level 0 the original problem, the first level of reformulation will consist of two problems with $\frac{n}{2}$ constraints each, the second level of four problems with $\frac{n}{4}$, and so on for a maximum of $\log_2 n = w$ levels, when we have only one constraint per block.

Sub-problem Tree (SP-T in the following) A tree structure is then necessary to implement the Recursive DWR and it is used for storing information about the different levels of the decomposition. The number of levels w to apply and the way of decomposing the problem are both fixed parameters that the algorithm takes as input. The preprocessing phase consists in creating $\sum_{k=0}^{w-1} 2^k$ "master problems" and n "leaf" problems, and then initializing them with a feasible solution. The root node contains the original master problem. The two child nodes of the root contain the associated pricing problem, they are again solved via DWR and hence two second-level master problems are associated to them. The same applies to all other levels of the SP-T, except for the last level. If the last level consists of just one constraint then the problem reduces to a KP, otherwise, a MIP solver can be used for solving the pricing problem. Note that in each pricing problem the objective function depends on the dual variables Π of its father node.

The Recursive DWR method can be implemented using two procedures: `B_P` and `DW_BOUND` (described in Algorithm 1 and Algorithm 2, respectively) that are recursively calling each other:

- `B_P(UB, IndNode,  $\bar{R}$ )`: this recursive depth first search procedure is used for solving via B&P the original problem or any pricing problem. The procedure receives in input the current best upper bound `UB`, the current node `IndNode` of the SP-T and a vector $\bar{R}$ of costs (i.e., the original objective function coefficients for the first call and the dual variables values Π in the procedure `DW_BOUND`). The procedure returns the optimal solution and the optimal solution value, or $(\emptyset, \text{UB})$ if the current best bound `UB` can not be improved. The following additional procedures are used:
 - `PROB(IndNode)`: this procedure returns the problem associated to a given node `IndNode`.
 - `INT_CHECK(Sol)`: this procedure returns `TRUE` if `Sol` is integer, `FALSE` otherwise.
 - `X_TO_FIX()`: this procedure returns the variable selected for branching.
 - `FIX_VAR(IndNode,  $\bar{X}_i$ , Value)`: this procedure fixes to `Value` (i.e., 0 or 1) the variable $\bar{X}_i$ on the problem `PROB(IndNode)` and its child nodes in the SP-T.

- **FREE_VAR**(IndNode, $\bar{X}_i$): this procedure resets lower and upper bounds of the variable $\bar{X}_i$ to their original values.
- **DW_BOUND**(IndNode, $\bar{R}$): this procedure is used to solve to optimality the LP relaxation of the master problem associated to the node IndNode of the SP-T, where $\bar{R}$ denotes the objective function coefficients associated with IndNode (i.e., the dual variables values Π except for the first call, in which they correspond to the original objective function coefficients). It returns the optimal solution and the corresponding value; and it uses the following additional procedures:
 - **TYPE**(IndNode): this procedure is used for checking whether IndNode is a LEAF of the SP-T.
 - **SET_OBJ_COEFS**($\bar{R}$, **PROB**(IndNode)): this procedure is used for setting the objective function coefficients of **PROB**(IndNode) to $\bar{R}$.
 - **SOLVE_LP**(**PROB**(IndNode)): this procedure is used for solving the LP relaxation of **PROB**(IndNode). It returns the primal and dual optimal solutions and their value.
 - **SOLVE_LEAF**(**PROB**(IndNode)): this procedure is used for solving the problem **PROB**(IndNode) if IndNode is a LEAF, it returns the optimal solution and the optimal solution value for **PROB**(IndNode) (i.e., solving **PROB**(IndNode) as a MIP).
 - **SP_LEFT**(IndNode), **SP_RIGHT**(IndNode): this procedure is used to retrieve the left and right child node of IndNode in the SP-T.

The above described implementation is started by calling **B_P**(∞ , 0, P), where P is the original cost vector ($p_i, i = 1, \dots, n$) and 0 stands for the root node of the SP-T. Several nested B&P are then invoked, and, during the solution process, all the sub-problems of the SP-T are adjusted accordingly to the current fixing via the procedure **FIX_VAR**. Since at each level of the recursion the original variables are explicitly kept in the reformulation, i.e., we use the *explicit master format* (see [8] for further details), we can branch on the original variables. This is particularly effective since it preserves the sub-problem structure during the branching scheme. In our implementation, we decided to branch on the variable with the largest fractional value in the current linear programming relaxation. Finally, two computational techniques are used to speed up the convergence of the Recursive DWR: the variables generated at each iteration are stored, in order to potentially be reused at the next pricing iterations; and each master problem is initialized with a feasible solution.

2.1 Recursive DWR for general MIPS

The Recursive DWR method we propose, in principle, could be used to solve generic MIPS. However, this would ask to solve a number of non-trivial design decisions, hence our implementation and experiments are restricted to the TKP, for which the method works very well. This is due to the structure of the TKP, which limits the number of design decisions, as well as to the fact that the problem is well suited for applying DWR.

Algorithm 1: B_P

```
input : UB, IndNode,  $\bar{R}$ .  
output : either one optimal integer solution of  $\text{PROB}(\text{IndNode})$ , or  $(\emptyset, \text{UB})$  if no improved solution is  
found.  
  
// computing current lower bound via column generation  
(Sol, Val)  $\leftarrow$  DW_BOUND(IndNode,  $\bar{R}$ );  
  
// bounding  
if Val  $\geq$  UB then return  $(\emptyset, \text{UB})$ ;  
;  
  
// updating feasible solution  
if INT_CHECK(Sol) then return (Sol, Val) ;  
;  
  
 $\bar{X}_i = \text{X\_TO\_FIX}()$ ;  
BestIntSol  $\leftarrow \emptyset$ ;  
FIX_VAR(IndNode,  $\bar{X}_i$ , 1);  
  
// depth first search, right side  
(SolOutBP, ValOutBP)  $\leftarrow$  B_P(UB, IndNode,  $\bar{R}$ );  
if SolOutBP  $\neq \emptyset$  then BestIntSol  $\leftarrow$  SolOutBP; UB  $\leftarrow$  ValOutBP;;  
  
FREE_VAR(IndNode,  $\bar{X}_i$ );  
  
// bounding  
if Val  $\geq$  UB then return (BestIntSol, UB);  
;  
  
FIX_VAR(IndNode,  $\bar{X}_i$ , 0);  
  
// depth first search, left side  
(SolOutBP, ValOutBP)  $\leftarrow$  B_P(UB, IndNode,  $\bar{R}$ );  
if SolOutBP  $\neq \emptyset$  then BestIntSol  $\leftarrow$  SolOutBP; UB  $\leftarrow$  ValOutBP;;  
  
FREE_VAR(IndNode,  $\bar{X}_i$ );  
return (BestIntSol, UB);
```

Specifically, the advantages and drawbacks of the Recursive DWR method are:

- it potentially allows to compute very strong linear programming relaxation bounds, hence, problems may often be solved at the root node of a Branch-and-Price algorithm; for the TKP, the two-block bound is quite difficult to compute (sometimes impossible) within the framework presented in Caprara et al. [8] while it can be computed by using the Recursive DWR method;
- it implies a computational overhead for solving not only the original reformulation but also (recursively) the sub-problems through column generation and B&P;
- it has an efficacy strongly related to the capability to define a “good” decomposition for the MIP to be solved (including the subsequent column generation MIPs); this area of research was recently investigated in [3, 4];
- aside from the methodology of decomposition, in order to be effective, it requires to carefully determine the depth of the recursion to apply, i.e., when to stop the recursion and solve the resulting problem in a “traditional” fashion.

Algorithm 2: DW_BOUND

```
input : IndNode ,  $\bar{R}$ 
output : the optimal solution Sol of the current LP relaxation and the corresponding value Val.
SET_OBJ_COEFS( $\bar{R}$ , PROB(IndNode));
ColAdd  $\leftarrow$  TRUE;
// adding columns with negative reduced costs
while ColAdd = TRUE do
    ColAdd  $\leftarrow$  FALSE ;
    // solving the problem and getting the related information
    (Sol, II, Val)  $\leftarrow$  SOLVE_LP(PROB(IndNode));
    // solving left sub-problem
    if TYPE(SP_LEFT(IndNode)) = LEAF then
        (SolLeft, ULeft)  $\leftarrow$  SOLVE_LEAF(PROB(SP_LEFT(IndNode)));
    else (SolLeft, ULeft)  $\leftarrow$  B_P( $\infty$ , SP_LEFT(IndNode), II);
    if ULeft < 0 then add column(s) to PROB(IndNode); ColAdd  $\leftarrow$  TRUE;
    ;
    // solving right sub-problem
    if TYPE(SP_RIGHT(IndNode)) = LEAF then
        (SolRight, URight)  $\leftarrow$  SOLVE_LEAF(PROB(SP_RIGHT(IndNode)));
    else (SolRight, URight)  $\leftarrow$  B_P( $\infty$ , SP_RIGHT(IndNode), II) ;
    if URight < 0 then add column(s) to PROB(IndNode) ; ColAdd  $\leftarrow$  TRUE ;
    ;
end
return (Sol , Val);
```

In summary, for applying the Recursive DWR to other problems, several design issues must be addressed, and we expect the method to be effective for problems with structures similar to the one of the TKP. In the TKP the key element is that constraints matrix is sparse and each variables appears in a limited number of consecutive constraints, determining in this way relatively independent subproblems.

3 Computational Experiments on the Temporal Knapsack Problem

In this section, we show that for the TKP it is possible to deal with large sub-problems by recursively decomposing the sub-problems rather than solving them directly. Indeed, the Recursive DWR allows us to compute the two-block bound, i.e., the linear programming relaxation of the DWR in which the constraints are split into two blocks only. For our experiments we consider some of the hardest odd-numbered TKP instances from Caprara et al. [8], namely we select instances from the first instance class and groups from V to X. All computational tests were conducted by using 1 core of a core i5-650 at 3.20GHz with 8Gb RAM, under a linux operating system. The general-purpose solver used was IBM ILOG CPLEX 12.3, with the default parameter setting. A recursive depth-first exploration strategy is used and branching is performed on the original x variables.

In a first set of experiments, we study the impact of the number of levels of recursion on the

Rows	level of recursion – # time limits							level of recursion – average time						
	1	2	3	4	5	6	7	1	2	3	4	5	6	7
768	0	0	0	0	0	0	0	211.9	151.3	182.5	182.8	266.1	370.8	1369.0
1024	1	0	0	0	0	0	2	1212.4	896.9	724.2	392.2	348.9	766.5	1943.2
1280	3	0	0	1	1	0	2	2549.9	860.9	1206.2	1343.2	1346.4	923.7	2109.9
1536	2	1	0	2	2	2	2	2175.8	1900.2	1521.4	1927.4	2545.5	2275.2	2748.7
1792	4	3	3	0	0	0	1	2992.2	2716.0	2643.7	1707.1	1057.5	1013.0	2158.6

Table 1: Impact of the level of recursion on the computing time for the two-block bound.

cons	max leaf size					
	50	100	150	200	250	900
768	0	0	0	0	0	0
1024	0	0	0	0	0	1
1280	1	1	1	0	0	3
1536	2	2	2	0	0	2
1792	0	0	0	0	3	4

Table 2: Number of time limits in function of the maximum number of constraints for the leaf subproblems.

computational effectiveness of the method when computing the two-block bound by allowing up to 1 hour of computing time. We evaluate the performance first by counting the number of time limits that are reached when computing the two-block bound and secondly by the average computing time (the averages include the time limits of 1 hour). We group together instances with the same number of constraints thus forming 5 homogeneous subsets. In Table 1, we report the number of time limits and the average computing times, obtained by stopping the recursion after a specified number of levels. The table indicates that the number of levels and the computing time are correlated. When the number of levels is low (i.e., up to 5–6) the average computing time decreases, while for larger number of levels the computing time increases. The initial reduction of computing time is due to the fact that the sub-problems become smaller and thus easier to solve while the additional effort of optimizing over a larger number of levels remains low. This trend inverts when the number of levels increases too much and the number of sub-problems becomes too large. From the table it emerges that the best choice for the recursion level depends on the size of the instances, i.e., in order to avoid time limits, 3 levels are necessary for instances with up to 1536 rows, and 4 levels are instead necessary for instances with 1792 rows. To clarify this issue we investigate the size of the *leaf* sub-problems that are then solved directly with CPLEX.

The previous choice corresponds to stop the recursion when the sub-problems have less than 200 constraints. In Table 2 we report the number of reached time limits according to different maximum leaf size.

In Table 3 we report a comparison between three different approaches to solve the TKP, i.e., the CPLEX MIP solver directly applied to model (1), the algorithm presented in Caprara et al. [8] (denoted as CFM in the following) and the Recursive DWR method. For a fair comparison, we test CFM and Recursive DWR method with two blocks, obtaining in this way the same root node bound for both methods, i.e., the two-block bound. The value of the obtained bound is

Instance	var	cons	CPLEX		CFM [8] Algorithm			Recursive DWR Method				
			time	gap	vars _{leaf}	cons _{leaf}	time _{root}	levels	vars _{leaf}	cons _{leaf}	time _{root}	time
I41	2071	768	565.6	-	1036	384	132.6	3	518	192	93.5	93.5
I43	2756	1024	TL	0.12	1378	512	TL	3	345	128	978.5	1107.4
I45	3433	1280	TL	0.35	1717	640	TL	3	429	160	258.4	258.4
I47	4154	1536	2902.5	-	2077	768	1,734.3	3	519	192	303.2	303.2
I49	4797	1792	TL	0.46	2399	896	TL	4	300	112	2322.1	2322.1
I61	2071	768	802.8	-	1036	384	202.4	3	518	192	190.5	190.5
I63	2763	1024	2627.2	-	1382	512	785.2	3	345	128	897.8	897.8
I65	3434	1280	TL	0.17	1717	640	TL	3	429	160	1823.4	3430.7
I67	4164	1536	TL	0.17	2082	768	TL	3	521	192	2523.9	2523.9
I69	4786	1792	2703.4	-	2393	896	TL	4	299	112	1510.7	1510.7
I71	2916	768	80.1	-	1458	384	73.8	3	729	192	69.9	69.9
I73	3832	1024	294.6	-	1916	512	417.7	3	479	128	276.9	276.9
I75	4771	1280	TL	0.04	2386	640	1,428.0	3	596	160	395.5	395.5
I77	5793	1536	TL	0.07	2897	768	754.9	3	724	192	949.0	949.0
I79	6800	1792	299.0	-	3400	896	622.5	4	425	112	1056.8	1056.8
I81	5210	768	1331.8	-	2605	384	437.1	3	1303	192	271.5	271.5
I83	6901	1024	55.11	-	3451	512	210.3	3	863	128	948.4	948.4
I85	8656	1280	1482.1	-	4328	640	532.9	3	1082	160	784.8	784.8
I87	10271	1536	TL	0.12	5136	768	TL	3	1284	192	961.7	961.7
I89	11992	1792	TL	0.08	5996	896	TL	4	750	112	1221.5	1221.5
I91	3117	768	205.1	-	1559	384	213.5	3	779	192	130.9	130.9
I93	4176	1024	2444.2	-	2088	512	1,052.7	3	522	128	390.2	390.2
I95	5209	1280	TL	0.06	2605	640	TL	3	651	160	1161.6	1161.6
I97	6215	1536	2027.4	-	3108	768	1,203.3	3	777	192	2869.5	2869.5
I99	7172	1792	TL	0.34	3586	896	TL	4	448	112	2424.1	2424.1

Table 3: Comparison between the different approaches.

then the same in the two cases, but CFM computes it by reformulating the problem into two sub-problems, which are then solved by **CPLEX**, while the Recursive DWR method iteratively reformulates the sub-problems as well, until subproblems have less than 200 constraints. The first three columns report the instance features, i.e., the name and the total number of variables and constraints. Then we report the time needed by **CPLEX** to solve the instances to proven optimality. If the time limit of 1 hour is reached, we report the percentage exit gap computed with respect to the best bound and the best heuristic solution value found. For the CFM algorithm, we report the average number of variables ($\text{vars}_{\text{leaf}}$) and constraints ($\text{cons}_{\text{leaf}}$) appearing in the final sub-problems (i.e., those solved by **CPLEX**) and the computing time to solve the root node. This table shows that quite often this method is not able to compute the two-block bound value within a time limit of one hour. Then the table reports the results of the Recursive DWR method; specifically the number of levels, the average number of variables and constraints for each sub-problem and the computing time. Finally the table reports the total computing time needed by the Recursive DWR method to perform the entire Branch-and-Price, i.e., solving the instances to optimality. In bold text, we report the best computing time among the tested algorithms. The Recursive DWR method outperforms the others in all except three cases in which **CPLEX** applied to the original formulation is the best option. It is worth noticing that in all cases, except two, the Recursive DWR method closes the instances at the root node (i.e., the root node solution is integer), in the remaining two cases the root node gap is less than 0.018%.

In Table 4, we report additional computational details for the instances considered. The first

Instance	opt	LP gap	time	ROOT gap	time	DW gap	time	vars	#lp	#ilp
I41	30866	14.024	0.0	0.865	1.5	0.000	92.2	142	231	386
I43	40934	13.502	0.0	1.113	1.8	0.018	978.5	381	669	1024
I45	50324	13.585	0.0	1.077	3.2	0.000	258.4	108	149	248
I47	59255	14.040	0.0	0.753	2.9	0.000	303.2	160	242	368
I49	69530	13.400	0.0	1.077	4.0	0.000	2322.1	1039	4479	6754
I61	71998	11.561	0.0	0.585	2.4	0.000	275.7	151	237	412
I63	97056	11.877	0.0	0.382	3.1	0.000	897.8	125	385	634
I65	120505	12.238	0.0	0.502	5.3	0.004	1823.4	415	643	1074
I67	142486	12.310	0.0	0.541	6.2	0.000	2523.9	211	305	534
I69	165076	12.245	0.0	0.658	7.7	0.000	1510.7	395	712	1086
I71	40982	11.322	0.0	0.554	1.3	0.000	118.3	192	225	374
I73	52447	12.138	0.0	0.612	1.8	0.000	276.9	152	243	404
I75	66179	12.113	0.0	0.678	2.4	0.000	395.5	104	205	332
I77	81982	11.320	0.0	0.575	3.2	0.000	949.0	177	327	536
I79	95037	11.599	0.0	0.562	3.9	0.000	1056.8	807	1992	2894
I81	71426	11.687	0.0	0.483	2.3	0.000	253.6	119	213	308
I83	96115	10.845	0.0	0.314	2.5	0.000	948.4	154	438	604
I85	119233	11.549	0.0	0.324	4.0	0.000	784.8	183	233	388
I87	142056	10.683	0.0	0.479	5.2	0.000	961.7	148	215	334
I89	167916	10.753	0.0	0.423	7.2	0.000	1221.5	322	709	970
I91	42685	11.950	0.0	0.856	1.7	0.000	172.7	139	212	352
I93	54437	11.538	0.0	0.666	1.9	0.000	390.2	195	331	498
I95	68432	10.958	0.0	0.676	2.8	0.000	1161.6	331	478	742
I97	80122	11.823	0.0	0.640	3.5	0.000	2869.5	650	1231	1852
I99	92380	12.287	0.0	0.800	5.5	0.000	2424.1	880	2106	2894

Table 4: Additional computational details.

two columns report the instance name and its optimal solution value. Then we present three percentage gaps, the first two gaps (*LP* gap and *ROOT* gap) are the linear programming relaxation gap, and the root node relaxation (linear programming relaxation plus default cuts generated at the root node), respectively, obtained when tackling Model (1) directly with **Cplex**. The third gap (*DW* gap) is obtained by the two-block bound, computed by the Recursive DWR method. All the gaps have been computed using the following formula: $(\text{opt-bound})/\text{opt} \times 100$, where *opt* is the optimal value of the instance. We report beside each gap the corresponding computing time. From the table it clearly emerges that *LP* gap corresponds to a fast relaxation of “poor” quality while *ROOT* gap denotes a better relaxation which requires “some” additional computational effort. *DW* gap is by far the smallest one, the computational time to obtain the associated bound is the largest but the quality is sufficient to prove optimality in all except two instances. In the last part of the table we report some further details of the Recursive DWR method. We report the total number of variables generated during the optimization (*vars*). Finally the table reports the number of linear programming relaxations solved (*#lp*), i.e., the number of times a master problem has been solved and the number of integer programming problems solved (*#ilp*), i.e., the number of times a leaf problem has been solved. We report in bold text the instances solved at the root node by the Recursive DWR method. In summary, the Recursive DWR method has been able to outperform both **Cplex** and **CFM**, computationally proving that a suitable number of reformulation levels helps in better solving the instances of TKP.

In Table 5, we investigate what happens when the Recursive DWR method is used as a stand-alone

Instance	var	cons	CPLEX	Recursive DWR Method			
			time	levels	vars _{leaf}	cons _{leaf}	time
I1-32	115	32	0.1	5	30	1	119.1
I2-32	217	32	0.1	5	30	1	33.0
I3-32	138	32	0.1	5	35	1	118.3
I4-32	238	32	0.1	5	35	1	44.4
I5-32	153	32	0.1	5	40	1	141.6
I1-64	205	64	0.2	6	30	1	1315.0
I2-64	418	64	0.1	6	30	1	528.7
I3-64	258	64	0.9	6	35	1	TL
I4-64	454	64	0.7	6	35	1	552.1
I5-64	280	64	0.1	6	40	1	2631.0
I1-128	374	128	0.3	7	30	1	TL
I2-128	830	128	1.6	7	30	1	TL
I3-128	492	128	3.3	7	35	1	TL
I4-128	886	128	8.6	7	35	1	1621.8
I5-128	531	128	2.3	7	40	1	TL

Table 5: Comparison between CPLEX and the Recursive DWR method used as a complete MIP solver.

MIP solver, i.e., when the Recursive DWR method is iteratively applied until the sub-problems become one-constraint MIPs. For the considered TKP instances, the *leaf* sub-problems are actually KP01 problems, and are solved with the `combo` algorithm (see Martello et al. [14]). To perform the experiment we considered the first instance for each group of Table 3 and we kept the first 32, 64, and 128 constraints. In bold text we report the winning method; the meaning of the columns in the table is the same as in the previous one. For these instances the computing times are far from being competitive with other methods, showing the limit of the Recursive DWR method when used as a stand-alone solver for the TKP.

4 Conclusions

In this study we investigated the possibility to solve the Temporal Knapsack Problem by applying the DWR recursively to solve the column generation sub-problems as well, in what we defined a Recursive DWR method. For the TKP we showed that the Recursive DWR method can be effective in solving hard instances that could not be solved by state-of-the-art approaches or MIP solvers applied to the original formulation. The TKP belongs to a class of structured MIPs called dynamic MIP. Dynamic MIPs are an extension of classical MIPs by a time horizon and a time interval in which each variable is active (see Bergner et al. [3, 4]). Since the Recursive DWR method has been effective on the TKP, other problems belonging to the same class of dynamic MIPs are natural candidates for further applications of the method in future works. Finally, the effectiveness of the Recursive DWR method deeply relies on the quality of the decomposition. We are confident that advances in the research on the topic will make the Recursive DWR method applicable to broader categories of MIPs.

Acknowledgments To the great sadness of the other authors, Alberto Caprara prematurely passed away while working on this problem and after contributing, as it often happened, some powerful insights on its structure. Alberto will be missed by his coauthors of the present paper as an amazing scientist and great friend.

The authors thank two anonymous referees for their careful reading and insightful comments.

References

- [1] E. M. Arkin and E. B. Silverberg, Scheduling jobs with fixed start and end times. *Discrete Applied Mathematics*, 18 (1987) 1–8.
- [2] N. Bansal, A. Chakrabarti, A. Epstein, and B. Schieber. A quasi-PTAS for unsplittable flow on line graphs. In *Proceedings of the 38th Annual ACM Symposium on the Theory of Computing (STOC 2006)*, pages 721–729, 2006.
- [3] M. Bergner, A. Caprara, F. Furini, M. Lübbecke, E. Malaguti, E. Traversi, Partial convexification of general mips by Dantzig-Wolfe reformulation, in: *Integer Programming and Combinatorial Optimization*, Vol. 6655 of *Lecture Notes in Computer Science*, Springer Berlin Heidelberg, 2011, pp. 39–51.
- [4] M. Bergner, A. Caprara, A. Ceselli, F. Furini, M. Lübbecke, E. Malaguti, E. Traversi, Automatic Dantzig-Wolfe reformulation of mixed integer programs, *Mathematical Programming* (2014) 1–34. DOI 10.1007/s10107-014-0761-5
- [5] P. Bonsma, J. Schulz, and A. Wiese, A constant factor approximation algorithm for unsplittable flow on paths, *CoRR* (2011).
- [6] G. Calinescu, A. Chakrabarti, H.J. Karloff, and Y. Rabani, Improved approximation algorithms for resource allocation. *Proceedings of the 9th International Conference on Integer Programming and Combinatorial Optimization (IPCO 2002)*, Springer-Verlag, 2002, pages 401–414.
- [7] B. Chen, R. Hassin, and M. Tzur, Allocation of bandwidth and storage, *IIE Transactions* 24 (2002) 501–507.
- [8] A. Caprara, F. Furini, E. Malaguti, Uncommon Dantzig-Wolfe reformulation for the temporal knapsack problem, *INFORMS Journal on Computing* 25 (2013) 560–571.
- [9] G. Desaulniers, J. Desrosiers, M. M. Solomon, *Column Generation*, Springer-Verlag, New York, 2005.
- [10] A. Darmann, U. Pferschy, and J. Schauer, Resource allocation with time intervals, *Theoretical Computer Science* 411 (2010) 4217–4234.
- [11] F. Furini, Decomposition and reformulation of integer linear programming problems, *4OR* 10 (2012) 219–220.
- [12] C.R. Glassey, Nested Decomposition and Multi-Stage Linear Programs, *Management Science* 20 (1973) 282–292.
- [13] J.K. Ho, A.S. Manne, Nested decomposition for dynamic models, *Mathematical Programming* 1 (1974) 121–140.

- [14] S. Martello, D. Pisinger, P. Toth, Dynamic programming and strong bounds for the 0-1 knapsack problem, *Management Science* 45 (1999) 414–424.
- [15] F. Vanderbeck, L. Wolsey, Reformulation and decomposition of integer programs, in: M. Jünger, T. Liebling, D. Naddef, G. Nemhauser, W. Pulleyblank, G. Reinelt, R. G., L. Wolsey (Eds.), *50 Years of Integer Programming 1958-2008 From the Early Years to the State-of-the-Art*, Springer, Berlin Heidelberg, 2010, pp. 431–502.
- [16] R.J. Wittrock, Dual nested decomposition of staircase linear programs, *Mathematical Programming Studies* 24 (1985) 65–86.